

DEPARTMENT OF INFORMATION TECHNOLOGY

24ITT32 - PROGRAMMING IN JAVA

INDIVIDUAL PROJECT ON
ACADEMIC MANAGEMENT SYSTEM

P R O J E C T R E P O R T

Submitted by

VAISNAV V M - 25ITL194

JAVA ASSIGNMENT

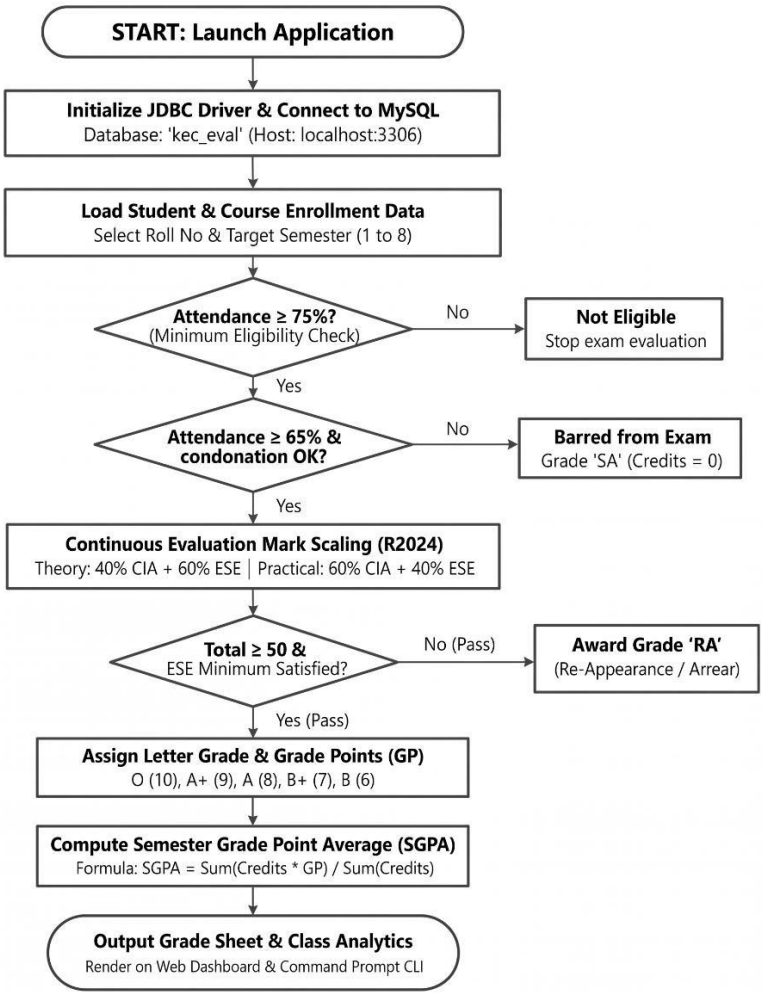
AIM:

Development of Kongu Engineering College (R2024) Student Academic & Continuous Evaluation System using Java JDBC.

OBJECTIVE:

Design and develop a database-driven Java application using JDBC to manage student records, perform incremental marks entry (Continuous Internal Assessments and End Semester Examinations), enforce attendance eligibility rules, and generate formal academic performance reports according to Kongu Engineering College (Autonomous) - Regulation 2024. System Requirements & Specifications.

FLOW CHART:



PROCEDURE:

- Load the MySQL Connector/J driver to enable communication between the Java application and the MySQL database.

- Establish a connection to the kec_eval database running on the local MySQL server using DriverManager.getConnection().

- Store and retrieve student details, including roll number, name, department, and semester, along with course information such as course code, title, credits, and course type.

- Enroll students in semester courses, record the number of classes held and attended, and calculate the attendance percentage.

- Verify examination eligibility based on attendance percentage. Students with 75% or above are eligible, those with 65%–74% require medical condonation verification, and those below 65% are marked as Shortage of Attendance (SA).

- Record CIA-1, CIA-2, and assignment marks, and calculate the scaled internal assessment marks according to the specified theory and practical course weightages.

- Record end-semester examination marks out of 100, verify the minimum passing requirements, and scale the marks according to the applicable theory and practical .

- Calculate the total marks by combining scaled CIA and ESE marks. Assign letter grades and grade points according to the specified grading scale, marking students as RA if they fail the examination or score below the required passing mark.

- Generate student grade sheets, attendance shortage reports, and department-wise pass percentage reports. Display the results through the interactive web dashboard and the command prompt console**.**

DATABASE SCHEMA:

1.STUDENTS TABLE:

Output | Result #2

Field	Type	Null	Key	Default	Extra
roll_no	varchar(15)	NO	PRI	NULL	
name	varchar(100)	NO		NULL	
department	varchar(20)	NO		NULL	
current_semester	int	NO		NULL	

2. COURSES TABLE:

Output | Result #2 | Result #3

Field	Type	Null	Key	Default	Extra
course_code	varchar(15)	NO	PRI	NULL	
course_title	varchar(100)	NO		NULL	
credits	int	NO		NULL	
course_type	enum('THEORY','PRACTICAL','EMBEDDED')	NO		NULL	

3. ENROLLMENTS TABLE:

Output | Result #2 | Result #3 | Result #4

Field	Type	Null	Key	Default	Extra
enrollment_id	int	NO	PRI	NULL	auto_increment
roll_no	varchar(15)	NO	MUL	NULL	
course_code	varchar(15)	NO	MUL	NULL	
semester	int	NO		NULL	
total_classes	int	NO		0	
attended_classes	int	NO		0	
medical_condonation	tinyint(1)	NO		0	

4. ASSESSMENT MARKS TABLE:

Output | Result #2 | Result #3 | Result #4 | **Result #5**

Field	Type	Null	Key	Default	Extra
enrollment_id	int	NO	PRI	NULL	
cia1	decimal(5,2)	YES		NULL	
cia2	decimal(5,2)	YES		NULL	
assignment_marks	decimal(5,2)	YES		NULL	
quiz_marks	decimal(5,2)	YES		NULL	
lab_marks	decimal(5,2)	YES		NULL	
ese_marks	decimal(5,2)	YES		NULL	
ese_absent	tinyint(1)	NO		0	
cia3	decimal(5,2)	YES		NULL	

SOURCE CODE:

MAINAPP.JAVA:

```
package com.kec.academic;

import java.sql.SQLException;
import java.util.Scanner;

public class MainApp {

    private static final Scanner scanner = new Scanner(System.in);

    private static final StudentDAO studentDAO = new StudentDAO();
    private static final CourseDAO courseDAO = new CourseDAO();
    private static final EnrollmentDAO enrollmentDAO = new EnrollmentDAO();
    private static final AttendanceDAO attendanceDAO = new AttendanceDAO();
    private static final AssessmentDAO assessmentDAO = new AssessmentDAO();
    private static final GradeDAO gradeDAO = new GradeDAO();
    private static final GradeResultDAO gradeResultDAO = new GradeResultDAO();
    private static final SGPACalculator sgpaCalculator = new SGPACalculator();
    private static final FinalGradeService finalGradeService = new FinalGradeService();
    private static final AssessmentWeightageMenu weightageMenu =
            new AssessmentWeightageMenu();

    public static void main(String[] args) {
        System.out.println("KEC ACADEMIC PERFORMANCE MANAGEMENT
SYSTEM");

        while (true) {
            displayMenu();
            System.out.print("Enter your choice: ");
            String choice = scanner.nextLine().trim();
```

```
try {
            switch (choice) {
                case "1" -> studentMenu();
                case "2" -> courseMenu();
                case "3" -> enrollmentMenu();
                case "4" -> attendanceMenu();
                case "5" -> assessmentMenu();
                case "6" -> gradeReportMenu();
                case "7" -> calculateSGPA();
                case "8" -> weightageMenu.showMenu(scanner);
                case "0" -> {
                    System.out.println("Thank you for using KEC Academic System!");
                    scanner.close();
                    return;
                }
                default -> System.out.println("Invalid choice!");
            }
        } catch (SQLException e) {
            System.out.println("Database operation failed: " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Operation failed: " + e.getMessage());
        }
    }
}

private static void displayMenu() {
    System.out.println("""

            ---------- MAIN MENU ----------
            1. Student Management
            2. Course Management
            3. Enrollment Management
            4. Attendance Management
            5. Assessment Management
            6. View Grade Report
            7. Calculate SGPA
            8. Assessment Weightage Configuration
            0. Exit
            """);
}

private static void studentMenu() throws SQLException {
    System.out.println("1. Add Student  2. View Students");
    System.out.println("3. Search Student  4. Update Student");
    System.out.println("5. Delete Student  0. Back");
    String choice = scanner.nextLine().trim();

    switch (choice) {
        case "1" -> addStudent();
        case "2" -> studentDAO.viewAllStudents();
        case "3" -> studentDAO.searchStudent(readText("Roll number: "));
        case "4" -> updateStudent();
```

```
  case "5" -> deleteStudent();
            default -> { }
        }
    }
    private static void addStudent() throws SQLException {
        String roll = readText("Roll number: ");
        String name = readText("Student name: ");
        String dept = readText("Department: ");
        int sem = readInteger("Semester: ");
        studentDAO.addStudent(roll, name, dept, sem);
    }

    private static void updateStudent() throws SQLException {
        String roll = readText("Roll number: ");
        String name = readText("New name: ");
        String dept = readText("New department: ");
        int sem = readInteger("New semester: ");
        studentDAO.updateStudent(roll, name, dept, sem);
    }

    private static void deleteStudent() throws SQLException {
        String roll = readText("Roll number: ");
        if (readText("Confirm deletion (yes/no): ").equalsIgnoreCase("yes"))
            studentDAO.deleteStudent(roll);
    }

    private static void courseMenu() throws SQLException {
        System.out.println("1. Add Course  2. View Courses");
        System.out.println("3. Search Course  0. Back");
        switch (scanner.nextLine().trim()) {
            case "1" -> courseDAO.addCourse(
                    readText("Course code: "), readText("Course title: "),
                    readInteger("Credits: "), readText("Type: ").toUpperCase());
            case "2" -> courseDAO.viewAllCourses();
            case "3" -> courseDAO.searchCourse(readText("Course code: "));
            default -> { }
        }
    }

    private static void enrollmentMenu() throws SQLException {
        System.out.println("1. Enroll Student  2. View Enrollments  0. Back");
        switch (scanner.nextLine().trim()) {
            case "1" -> enrollmentDAO.enrollStudent(
                    readText("Roll number: "), readText("Course code: "),
                    readInteger("Semester: "));
            case "2" -> enrollmentDAO.viewEnrollments();
            default -> { }
        }
    }

    private static void attendanceMenu() throws SQLException {
        System.out.println("1. Update Attendance  2. Update Condonation");
```

```
System.out.println("3. View Attendance Report  0. Back");
    switch (scanner.nextLine().trim()) {
        case "1" -> attendanceDAO.updateAttendance(
                readInteger("Enrollment ID: "),
                readInteger("Total classes: "),


                readInteger("Attended classes: "));
        case "2" -> attendanceDAO.updateCondonation(
                readInteger("Enrollment ID: "),
                readText("Approved? (yes/no): ").equalsIgnoreCase("yes"));
        case "3" -> attendanceDAO.viewAttendanceReport();
        default -> { }
    }
}

private static void assessmentMenu() throws SQLException {
    System.out.println("1. Enter Marks  2. View Assessments  0. Back");
    if (scanner.nextLine().trim().equals("2"))
        assessmentDAO.viewAssessments();
    else
        System.out.println("Use the full assessment input implementation to enter marks.");
}

private static void gradeReportMenu() throws SQLException {
    System.out.println("1. Student Grade Details");
    System.out.println("2. Saved Final Grades");
    System.out.println("3. Calculate and Save Grades");
    String roll = readText("Roll number: ");

    switch (scanner.nextLine().trim()) {
        case "1" -> gradeDAO.viewStudentGradeDetails(roll);
        case "2" -> gradeResultDAO.viewGrades(roll);
        case "3" -> finalGradeService.calculateGradesForStudent(roll);
        default -> { }
    }
}

private static void calculateSGPA() throws SQLException {
    String roll = readText("Roll number: ");
    int sem = readInteger("Semester: ");
    sgpaCalculator.calculateSGPA(roll, sem);
}

private static String readText(String prompt) {
    System.out.print(prompt);
    return scanner.nextLine().trim().toUpperCase();
}

private static int readInteger(String prompt) {
    while (true) {
        try {
```

```
System.out.print(prompt);
                return Integer.parseInt(scanner.nextLine().trim());
            } catch (NumberFormatException e) {
                System.out.println("Enter a valid whole number.");
            }
        }
    }
```

DB CONNECTION.JAVA:

```
package com.kec.academic;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class DBConnection {

    private static final String URL =
            "jdbc:mysql://localhost:3306/kec_academics";

    private static final String USER = "root";

    public static Connection getConnection() throws SQLException {

        String password = System.getenv("KEC_DB_PASSWORD");

        if (password == null || password.isBlank()) {
            throw new SQLException(
                "Database password not found. Set KEC_DB_PASSWORD first."
            );
        }

        return DriverManager.getConnection(URL, USER, password);
    }
}
```

OUTPUT SCREENSHOTS:

MAIN MENU:

```
==========================================
                MAIN MENU
==========================================
1. Student Management
2. Course Management
3. Enrollment Management
4. Attendance Management
5. Assessment Management
6. View Grade Report
7. Calculate SGPA
8. Assessment Weightage Configuration
0. Exit
==========================================
Enter your choice:
```

STUDENT & COURSE MANAGEMENT:

1.STUDENT REGISTRATION:

```
---------- STUDENT MANAGEMENT ----------
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
0. Back to Main Menu
Enter choice: 1
Enter roll number: 25ITR114
Enter student name: KAMALESH . S
Enter department: IT
Enter current semester: 3
Student added successfully!
```

2. COURSE CATALOG:

```
---------- COURSE MANAGEMENT ----------
1. Add Course
2. View All Courses
3. Search Course
0. Back to Main Menu
Enter choice: 1
Enter course code: 23ITT31
Enter course title: C++
Enter course credits: 4
Course Types: THEORY, PRACTICAL, EMBEDDED
Enter course type: EMBEDDED

Course added successfully!
```

3. COURSE ENROLLMENT & ATTENDANCE TRACKING:

```
---------- ENROLLMENT MANAGEMENT ----------
1. Enroll Student in Course
2. View Enrollment Records
0. Back to Main Menu
Enter choice: 1
Enter student roll number: 25ITR111
Enter course code: 23ITT31
Enter semester: 3
Student enrolled successfully!
```

```
---------- ATTENDANCE MANAGEMENT ----------
1. Update Attendance
2. Update Medical Condonation
3. View Attendance Report
0. Back to Main Menu
Enter choice: 1
Enter enrollment ID: 3
Enter total classes conducted: 100
Enter classes attended: 60
Attendance updated successfully!
```

INCREMENTAL MARKS ENTRY:

```
---------- ASSESSMENT MANAGEMENT ----------
1. Enter or Update Assessment Marks
2. View Assessment Records
0. Back to Main Menu
Enter choice: 1
Enter enrollment ID: 3
Enter marks from 0 to 100. Leave blank if not entered.
CIA 1 marks: 75
CIA 2 marks: 80
CIA 3 marks: 90
Assignment marks: 20
Quiz marks: 20
Lab marks: 20
ESE marks: 20
Was the student absent for ESE? (yes/no): NO
Assessment marks saved successfully!
```

ATTENDANCE & GRADING RULES:

1.ATTENDANCE FILTER:

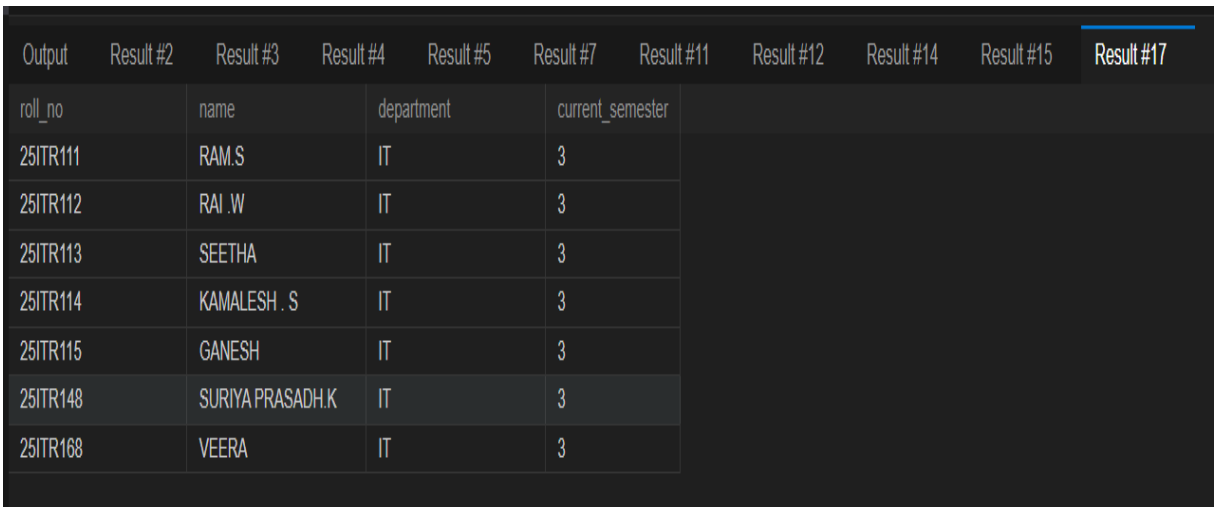
```
========================================
Student Roll No : 25ITR168
Student Name    : VEERA
Course Code     : 24ITC30
Course Title    : C
Course Type     : PRACTICAL
Attendance      : 60.00%
Medical Condonation Approved: true
CIA 1           : 75.00
CIA 2           : 80.00
CIA 3           : 90.00
Assignment Marks : 20.00
Quiz Marks      : 20.00
Lab Marks       : 20.00
ESE Marks       : 20.00
ESE Absent      : false
Final Grade     : Not calculated yet
========================================
```

STUDENT MARK :

```
---------- GRADE REPORT ----------
1. View Student Grade Details
2. View Saved Final Grades
3. Calculate and Save Final Grades
0. Back to Main Menu
Enter choice: 3
Enter student roll number: 25ITR115
Grade result saved successfully!
23ITT31 | Internal: 64.20 | ESE: 25.00 | Aggregate: 57.10 | Grade: B | Grade Point: 6 | Status: PASS
```

DATABASE:

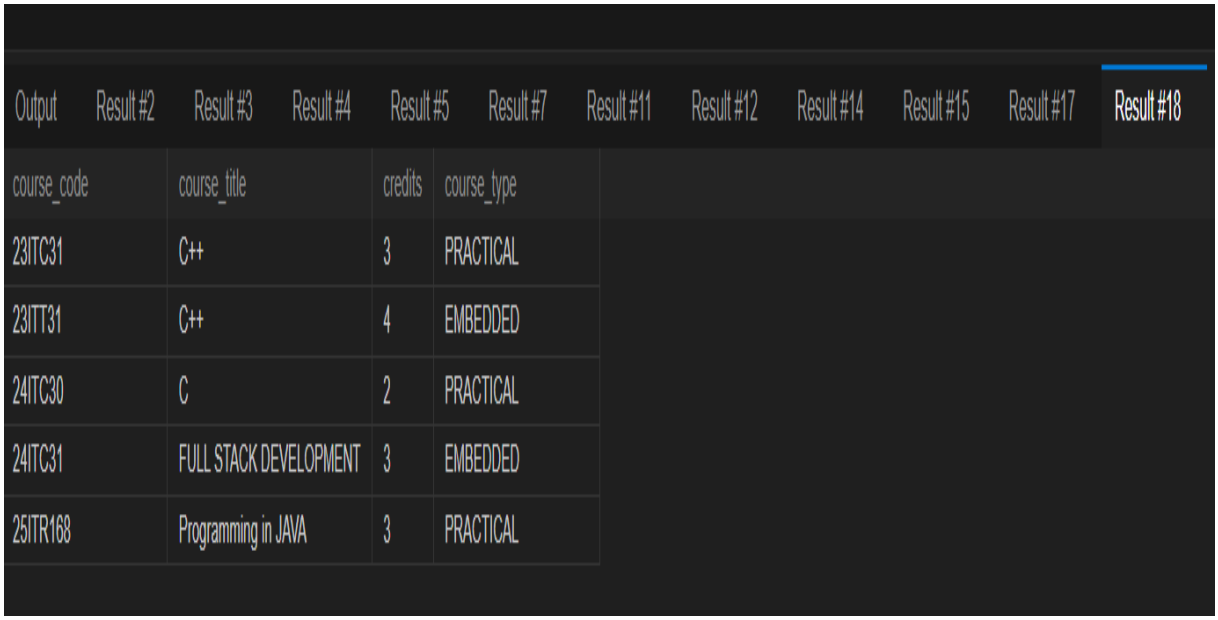
STUDENTS DATABASE:



Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #7 | Result #11 | Result #12 | Result #14 | Result #15 | Result #17

roll_no	name	department	current_semester
25ITR111	RAM.S	IT	3
25ITR112	RAI .W	IT	3
25ITR113	SEETHA	IT	3
25ITR114	KAMALESH . S	IT	3
25ITR115	GANESH	IT	3
25ITR148	SURIYA PRASADH.K	IT	3
25ITR168	VEERA	IT	3

COURSES DATABASE:



Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #7 | Result #11 | Result #12 | Result #14 | Result #15 | Result #17 | Result #18

course_code	course_title	credits	course_type
23ITC31	C++	3	PRACTICAL
23ITT31	C++	4	EMBEDDED
24ITC30	C	2	PRACTICAL
24ITC31	FULL STACK DEVELOPMENT	3	EMBEDDED
25ITR168	Programming in JAVA	3	PRACTICAL

ENROLLMENTS DATABASE:

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #7 | Result #11 | Result #12 | Result #14 | Result #15 | Result #17 | Result #18 | Result #19

enrollment_id	roll_no	course_code	semester	total_classes	attended_classes	medical_condonation
3	25ITR168	24ITC30	3	100	70	true
4	25ITR148	24ITC31	3	0	0	false
6	25ITR111	23ITT31	3	0	0	false
8	25ITR112	23ITC31	3	0	0	false
10	25ITR113	23ITC31	3	40	36	false
11	25ITR115	23ITT31	3	40	36	false

RESULT:

Thus, the Student Academic and Continuous Evaluation System for Kongu Engineering College (Autonomous) Regulation 2024 was successfully designed, developed, and tested using Java (JDBC) and MySQL. The system accurately automates the calculation of Continuous Internal Assessments (CIA), End Semester Examinations (ESE), Letter Grades, Attendance Eligibility, and Semester Grade Point Average (SGPA).

REFERENCES:

1. Oracle, Java SE Documentation – JDBC Overview and Architecture. https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/ .

2. Oracle, Java SE API – HttpServer. https://docs.oracle.com/en/java/javase/17/docs/api/jdk.httpserver/com/sun/net/httpserver/HttpServer.html .

3. Oracle, MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/ .

4. Oracle, MySQL Connector/J Developer Guide. https://dev.mysql.com/doc/connector-j/8.0/en/ .

5. University Grants Commission, Guidelines for Choice Based Credit System (CBCS). https://www.ugc.gov.in/pdfnews/8023719_Guidelines-for-CBCS.pdf .

6. Chart.js, Chart.js Documentation. https://www.chartjs.org/docs/latest/ .

7. Tailwind CSS, Tailwind CSS Documentation. https://tailwindcss.com/docs .

8. GeeksforGeeks, Establishing JDBC Connection in Java. https://www.geeksforgeeks.org/establishing-jdbc-connection-in-java/ .

9. YouTube, Java JDBC with MySQL Tutorial Videos. https://www.youtube.com/results?search_query=Java+JDBC+MySQL+tutorial .

10. Anthropic, Claude AI Assistant (used for project ideas and guidance). https://claude.ai/ .

GITHUB:

LINK:	